Tenderly to TenderOS Concept Map

A feature mapping and project development framework

Prepared 19 October 2026 | TenderOS means the TenderProof OS / ProposalGuard local-first diagnostic MVP in this workspace.

Purpose and design conclusion

Tenderly is a useful product architecture reference because it unifies a working console, scenario simulation, monitoring and response, and infrastructure around one operational record. TenderOS can adapt that product logic to tender decisions: organize each opportunity as a case, model bid scenarios before commitment, inspect source-to-recommendation evidence, monitor deadlines and unresolved conditions, and produce governed outputs. The domains are different: Tenderly manages on-chain systems; TenderOS manages tender/proposal work. The mapping transfers product patterns, not blockchain features.

The existing MVP already has a strong case workflow and evidence-centered analysis. The recommended direction is to evolve it into a case operating layer with controlled scenario planning, traceable analysis, deadline/gap monitoring, and a reusable knowledge layer. Treat proposed capabilities as roadmap concepts, not as currently implemented functions.

Mapping status: **Implemented** **Partial** **Proposed** **Not applicable**

Feature by feature mapping

Status reflects the repository as inspected for this concept map. “Partial” means a related workflow or record exists, while the Tenderly pattern still needs a deliberate end-to-end implementation.

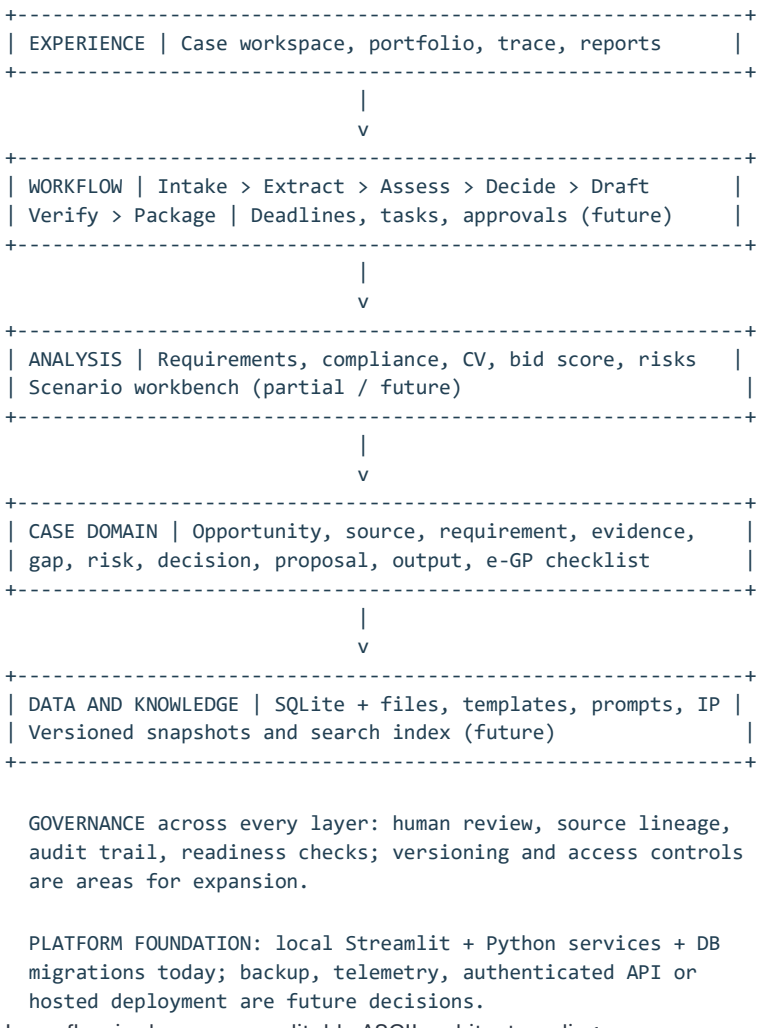
Tenderly capability	TenderOS equivalent concept	Status	TenderOS fit and development note
Tenderly Console: shared operational workspace, projects and workspaces	TenderOS Case Workspace: client, tender, donor, country, sector, deadline; case status and coordinated diagnostic pages.	**Implemented**	TenderOS already centers work on diagnostic cases. Team collaboration and multi-user access are outside the local MVP.
Explorer: unified search across transactions, wallets, contracts and chains	TenderOS Opportunity and Case Explorer: search cases, source files, requirements, evidence, outputs, and reusable IP.	**Partial**	Case lists and case selection exist; cross-entity search, facets, saved views, and global navigation are roadmap items.
Decoded activity timeline and transaction overview	Case Activity Timeline: source intake, extraction, edits, approvals, score changes, verification and export events.	**Partial**	AuditEvent exists; a complete human-readable event timeline and object-level provenance view would extend it.
Debugger: step through execution and locate exact failure	Evidence Trace / Decision Inspector: trace each recommendation or readiness flag back through rule, requirement, source page/section, evidence and reviewer.	**Proposed**	Build explainable, source-linked decisions; do not imply that this is a literal code debugger.
Full error and stack traces with readable messages	Diagnostic Exceptions and Extraction Review: actionable parse/OCR/validation errors with source location, remedy and reviewer disposition.	**Partial**	Source handling and extraction-failure gap recording exist; consistent error taxonomy and inline remediation are future work.

Tenderly capability	TenderOS equivalent concept	Status	TenderOS fit and development note
Custom views, filters, grouping, address labels and tags	Case portfolio views, filters and labels: sector, donor, deadline, readiness, lead status, risk, and owner.	Partial	Case metadata and lead concepts exist; portfolio-level configurable views and tagging are roadmap items.
Shared teams, comments and collaboration	Reviewer assignments, comments, decisions, and controlled hand-offs across proposal, technical, finance and leadership roles.	Proposed	Current application is explicitly local-first/internal with no multi-tenant auth or team collaboration.
Transaction Simulator UI: vary parameters and rerun before committing	Bid Scenario Workbench: compare pursue/no-bid, partner/subconsultant, expert-team, price/budget, schedule and evidence-collection scenarios.	Partial	Bid/no-bid scorecard and recommendation exist; scenario comparison, assumptions and saved variants are not implemented.
Simulation API and reusable simulation calls	Rules and Analysis Service: deterministic scoring, requirement normalization, readiness checks, and scenario calculations callable from UI and future integrations.	Partial	Services exist for scoring, normalization and verification; there is no public API or external AI service connected.
Simulation bundles: sequence multiple actions atomically	Bid Plan / Scenario Bundle: model linked changes together (for example, partner + expert substitution + schedule adjustment) and show combined effects.	Proposed	Treat as a planning workspace with explicit assumptions; never silently change the submitted baseline.
Virtual Environments: fork production state, override and replay history	Case Sandbox / Versioned Working Copy: duplicate a case baseline, test alternative compliance/evidence/methodology plans, compare with the source case.	Proposed	Local case data and deterministic anchors provide a base. Branching, versioning, diff and merge controls are new.
Historical replay from any past block; fork saved simulation	Case Snapshots and Replay: reproduce a prior recommendation from its source versions, scoring rules, evidence and timestamp.	Proposed	Audit events exist, but immutable snapshots of inputs, rule versions and reproducible scoring are not established.
Public Virtual Environment explorer for auditors	Read-only Client/Audit Package: share a scoped, versioned diagnostic with traceable sources and clear readiness labels.	Partial	Reports/workbooks and output package records exist; secure sharing, access scopes and reproducibility are future work.
Transaction traces, state changes and fund-flow visualization	Requirement-to-Evidence Traceability: navigate ToR clause -> normalized requirement -> compliance row -> evidence/gap -> risk -> proposal section -> output.	Partial	Stable match_key/canonical_anchor, compliance rows, gap items, risks and proposal skeleton exist; unified visual trace is a next step.
Gas profiler and performance optimization	Effort, Cost and Delivery Feasibility: expose budget attractiveness, staff loading, deadline feasibility, and effort assumptions in the decision record.	Partial	Scorecard covers budget/value and deadline feasibility; quantified effort/cost models are not documented as implemented.
Contracts and verified source metadata	Tender Source Register: retain ToR/amendment/addendum identity, upload, extracted text, date/version, section anchors and review state.	Partial	Sources and extracted text exist. Amendment comparison, provenance completeness and version conflict handling need development.
Contract read/write interfaces and protocol modules	Tender Rules and Donor Profiles: reusable eligibility/compliance rules, sector methodology modules, templates and known donor instructions.	Partial	Prompt/template/reference modules and reusable IP capture exist; governed rule packs and their lifecycle are future work.
Alerts on events, state changes, balances and function calls	Tender Alerts: deadline milestones, addenda, missing mandatory evidence, unresolved critical risks, stale inputs and readiness regression.	Partial	Workflow guidance, gaps/risks and e-GP stages exist; event-driven notification and escalation are not implemented.

Tenderly capability	TenderOS equivalent concept	Status	TenderOS fit and development note
Multichain monitoring with one rule	Portfolio Monitoring: apply shared control checks across all active cases and show exceptions by client, donor, country and deadline.	**Proposed**	The current MVP is case oriented and has no centralized portfolio monitoring layer.
Web3 Actions: automated code triggered by alert	Controlled Tender Actions: create tasks, draft clarification questions, request missing inputs, or prepare review queues after a trigger.	**Partial**	Clarification question and workflow optimization concepts exist. Human approval must gate external communications and consequential changes.
Alert routing to email, Slack, PagerDuty, webhooks	Notification Routing: in-app queue first; later authorized email/chat integrations with per-user preferences and delivery audit.	**Proposed**	No connector or notification routing is present in the local MVP.
Node RPC: production infrastructure, auth, rate limits and analytics	TenderOS Platform Foundation: local database/storage today; future authenticated service/API, backup, access control, integration and operational telemetry.	**Partial**	SQLite/local storage and local launcher are implemented. SaaS hosting, billing, multi-tenant identity and production SLA are explicitly out of scope.
CI/CD and staging on realistic state; simulation in scripts/bots	Release and Case Verification: deterministic checks for source coverage, mandatory criteria, risk/gap state, prohibited report language, and export readiness.	**Implemented**	Verification checks and readiness labels exist. Automated deployment pipelines and external integration tests are separate platform work.
Support operations: inspect a case and resolve issues with full context	Diagnostic Delivery Desk: triage blocked inputs, extraction issues, client questions, decision rationale and output revisions.	**Partial**	Workflow optimizer, audit log, clarification questions and reports support this pattern; service queue/ownership SLAs are not present.
Agentic workflows validated by simulation before execution	Evidence-grounded Assistance: optional AI may draft extraction or analysis; deterministic validation and human review control acceptance.	**Proposed**	Prompt files are present, but no external AI API or model is connected. AI output must remain a suggestion and cite supplied evidence.
Supported networks, chain-specific capability surface	Tender Context Packs: donor, procurement regime, country, sector, language/OCR and tender portal differences, with explicit applicability.	**Partial**	Bangladesh e-GP readiness and reference notes exist; broader donor/country packs and coverage controls are future work.

TenderOS layer architecture

The diagram adapts Tenderly's product architecture pattern while grounding every TenderOS layer in the current MVP or labeling it as future work. The proposal is a layered, case-centered system; it does not imply blockchain infrastructure.



Layer flow is shown as an editable ASCII architecture diagram.

Layer responsibilities and maturity

Layer	Responsibility	Current state / direction

Layer	Responsibility	Current state / direction
Experience	Provide clear case navigation, portfolio overview, workflow guidance and reviewable outputs.	Current case pages, workflow optimizer, reports and Excel export; cross-case views and trace visualization are planned.
Workflow orchestration	Move work through intake, analysis, decision, proposal, verification and packaging; surface next action and due date.	Core workflow guidance and e-GP checklist exist; assignment, approvals, reminders and escalations are future work.
Analysis and decision	Turn source material into structured requirements, evidence links, readiness, gaps, risks and decision options.	Extraction, compliance, CV matching, scorecard, risk/gap, clarification and verification services exist; scenario comparison is planned.
Case domain	Keep the tender opportunity as the shared object tying sources, criteria, evidence, people, decisions and outputs together.	SQLAlchemy models cover core case and diagnostic entities. Versioned baselines and formal decision records should be added deliberately.
Trust and governance	Make findings reproducible and reviewable; preserve source lineage, uncertainty, human acceptance and output readiness.	Anchors, audit events, human-review language, readiness states and prohibited-phrase checks exist; source page-level citations, rule versioning and access policies need expansion.
Data and knowledge	Store source files, extracted text, case records and reusable donor/sector/proposal intelligence.	Local SQLite/files and IP capture exist; indexing/search, retention controls, encryption and immutable snapshots are future work.
Platform and operations	Provide reliable local operation first, then add service controls only when justified by product scope.	Local Streamlit/Python/SQLite MVP exists. Authentication, multi-tenant SaaS, billing and external AI are intentionally not implemented.

Project development framework

Use the Tenderly pattern as a sequence of product questions: what must the operator see, what should be testable before commitment, what needs monitoring, and what foundation makes it dependable? For TenderOS, translate that sequence into a governed tender lifecycle and build in increments.

Phase	Work	Exit evidence	Design guardrail
0. Product boundary and evidence baseline	Confirm users, paid diagnostic service, case lifecycle, local-first deployment, supported tender contexts and exclusions. Capture existing screens, data entities, output examples and failure cases.	Approved concept, source-of-truth inventory, capability register with implemented/partial/proposed tags.	MVP scope already excludes public SaaS, billing, multi-tenant auth and connected AI.
1. Case console and source spine	Make case workspace the stable entry point. Improve source register, amendment/version handling, requirement anchors, global search and activity timeline.	Every finding points to a case, source version and requirement; users can find the latest authoritative input.	Preserve original files; flag OCR uncertainty and require human review.
2. Simulation and decision workbench	Add saved scenarios for bid/no-bid alternatives; make assumptions editable; calculate score/risk changes; compare variants to a locked baseline.	A reviewer can explain why recommendation changed and reproduce each scenario.	No scenario may overwrite evidence or imply certainty; show unknowns and sensitivity.
3. Traceable analysis and reusable modules	Build end-to-end requirement -> evidence -> gap/risk -> proposal section trace. Govern donor/context packs, methodology modules and reusable IP.	Coverage view; source-linked findings; reviewed reusable modules with owner, version and applicability.	Prevent unsupported claims; retain source context and avoid cross-client data leakage.
4. Monitoring and controlled actions	Introduce deadline and addendum tracking, missing-input alerts, stale-data signals and assigned tasks. Add in-app routing before external integrations.	Exceptions are visible, owned, due-dated and auditable; notification delivery can be reconciled.	External messages, client-facing changes and bid decisions remain human-approved.
5. Verification, packaging and operations	Expand reproducibility, role-based review, output versioning, backup/restore and operational checks. Add hosted/auth/API capability only if deployment model changes.	Repeatable client-ready package with readiness reason, audit trail, source trace and recovery procedure.	Keep local-only as default until security, privacy, support and operating model are specified.

Core product rules

- Evidence before assertion: each compliance conclusion, risk and CV claim should resolve to supplied evidence or be marked Gap/Pending Input.
- Scenarios are reversible: save assumptions and variants, show the baseline, and explain score changes instead of mutating the case silently.
- Human approval at consequential boundaries: bid/no-bid, client-facing language, evidence acceptance and external messages require a named reviewer.
- Reproducibility by default: preserve source versions, normalized requirements, rule versions, reviewer actions, readiness result and output package identity.
- Local-first remains a product constraint until a separate decision establishes access control, hosting, retention, support and privacy requirements.

Measures of progress

Track operational quality rather than win-rate claims: percentage of mandatory requirements linked to a source and evidence state; unresolved mandatory gaps at decision time; time from source intake to reviewed recommendation; extraction correction rate; stale-input/addendum acknowledgement time; verification failures before export; and the proportion of client-ready findings with a complete reviewer and source trail.

Sources and scope: Tenderly official documentation (https://docs.tenderly.co/introduction) and product overview (https://tenderly.co/pricing), checked 19 October 2026. TenderOS status is based on tenderproof_os/README.md, QUICKSTART.md, UI modules, models.py and services/ in this workspace; it describes this repository snapshot, not an independently deployed system.